

CartPole Repository Architecture

Purpose

This repo provides scripts so a user can train a cartpole model, simulate it on a GUI and convert the model using hls4ml before finally using Vitis/Vivado to deploy it on hardware.

Main Modes / Steps

- Data/Training (SI_Toolkit pipeline)
- Simulation (GUI + controllers + predictors)
- Conversion (hls4ml + Vivado reports)
- Physical control (PC runtime loop + serial interface)
- Build/deploy (Vivado/Vitis TCL + firmware)

Repo Structure (Stable)

Driver/:

- Role: Manages the PC side runtime control loop and hardware communication for the physical cartpole
- Dependencies: Control_Toolkit(ASF), SI_Toolkit (predictors/models), Experiment Workspace artifacts
- Purpose: Real time control, loads trained models and normalization data, and interfaces with the hardware over serial or other I/O

Driver/CartPoleSimulation: Implements the cartpole simulator, GUI, and sim control loop

- Dependencies: Control_Toolkit(ASF) controllers, SI_Toolkit(ASF) predictors, Experiment artifacts
- Purpose: Allows trained controllers and models to be tested in simulation, visualized and logged before deployment to hardware
- Upstream: Core CartPole dynamics, GUI framework and simulation infrastructure

SI_Toolkit + SI_Toolkit_ASF: (CartPoleSimulation Submodule)

- Role: Provides the machine learning pipeline for data generation, normalization, training, evaluation, and hls4ml conversion
- Dependencies: TensorFlow/ML libraries, Experiment Workspace, training and HLS configuration files
- Upstream: Core SI_Toolkit library code, model training logic, predictors, and framework abstractions
- Editable: SI_Toolkit_ASF, .yaml files, run scripts and experiment paths
- Purpose: Produces trained neural network models and normalization statistics, and converts selected models into HLS compatible representations for FPGA synthesis

Control_Toolkit + Control_Toolkit_ASF: (CartPoleSimulation Submodule)

- Role: Defines controller architectures, optimizers, and cost functions used in both simulation and physical control
- Dependencies: SI_Toolkit predictors/models, controller configuration YAML files
- Purpose: Encapsulates control logic (e.g., learned controllers, classical controllers) and provides an interface for simulation and execution
- Upstream: Core controller base classes and interfaces, generic optimizers and controller logic
- Editable: Control_Toolkit_ASF, .yaml files, CartPole specific controllers, Application specific cost functions and tuning

CartPole: (CartPoleSimulation Submodule)

- Role: Implements the physical CartPole environment used by both the GUI simulator and data/experiment generation
- Dependencies: Control_Toolkit controllers + environment base, SI_Toolkit predictors/models, YAML configuration files
- Purpose: Provide a single, reusable simulation source for CartPole's behavior

Experiment-1/: Seed Dataset (Static)

- Role: Central storage for all runtime and training artifacts shared across repo components
- Has Recordings (data to train/validate), Normalization Info & Models artifacts
- This is a read only template, all active runs operate on copies located under Driver/CartPoleSimulation/SI_Toolkit_ASF/Experiments/.
- **config_training_classic.yaml** (recipe for training the neural network)
 - Focused on stable, repeatable training runs
 - More validation to prevent mistakes (typos, normalization, etc)
- **config_hls.yaml** (Configurations for the HLS side of things)
 - Precisions, Strategy (default = resource), which backend, board, etc.
- **config_training_fast.yaml** (recipe for training the neural network0)
 - Minimal checks/overhead for quicker training
 - Assumes inputs/data paths are already correct

Release:

- Automates cloud lab and hosting demo environments
- Dependencies: Kubernetes tools, Xilinx tools, repo entry scripts
- Purpose: Lets users run training, simulation, and build workflows in a browser accessible hosted desktop environment

Model:

- Feed forward neural network that learns a control policy
- Given CartPole's current state (target), predicts the control action ($Q_{calculated}$)
- Tensorflow, dense, fully connected, 2 hidden layers 32 neurons, regression

FPGA Build Toolchain

Vivado: Used for hardware design / synthesis

- Uses the hls4ml output to synthesize FPGA logic
- Generates bitstream (.bit)
- Exports hardware platform (.xsa)

Vitis: Used for software / system integration

- Builds the embedded software that runs on the Zynq ARM CPU
- Compiles firmware and links it to the Vivado hardware
- Produces BOOT.bin

hls4ml Conversion Pipeline: Takes the trained neural network model and makes it a synthesizable HLS design

- Dependencies: SI_Toolkit_ASF conversion scripts, config_hls.yml, trained model artifacts, Vivado 2020.1 environment
- Purpose: Outputs synthesis reports and later generates the HLS project that is later synthesized into a FPGA bitstream using the build pipeline

FPGA Build Pipeline: Synthesizes, links, and packages FPGA firmware for deployment to Zynq later

- Dependencies: hls4ml output, Vivado, Vitis, hardware platform
- Purpose: Produces the final BOOT.bin image containing the bitstream and firmware for deploying on a SD card

Workflow / Student Steps (entry scripts)

Important

- All training simulation, conversion & runtime loads occur from this active workspace (Driver/CartPoleSimulation/SI_Toolkit_ASF/Experiments/Experiment-1), not the Experiment-1 seed folder

Environment Setup

Start.sh: Sets up a headless graphical desktop environment and exposes it over a browser via noVNC

- Logic: Runs all services in the background and exposes the desktop at port 6080 via a web interface

Setup_xilinx.sh: Sources the setting64 for Vitis/Vivado for later use

Install_zybo_board.sh: Installs the correct board

Entry Point Scripts

Step2-no-train:

- Logic: Creates directory to store experiments, copies experiment elements into it, sets paths
- Uses hardcoded experiment name

Step2.sh:

- Logic: Prepares an experiment, updates training configuration paths, and runs preprocessing and neural network training
- Uses hardcoded experiment name
- Execution: Runs A1_Create_Normalization_File.py, A2_Train_Network.py
 - A1_Create_Normalization_File.py:
 - Calls load_yaml to load a training configuration & calls calculate_normalization_info to compute normalization parameters for the dataset.
 - A2_Train_Network.py
 - Calls train_network()
- Copies the seed experiment into active workspace (Driver/CartPoleSimulation/SI_Toolkit_ASF/Experiments/Experiment-1)

Step3: Configures a CartPole simulation by updating a YAML config file, then launches a Python GUI.

- Inputs: Accepts an optional model name argument; otherwise uses a default neural network model name.
- Logic: Detects OS for correct syntax, updates four fields in config_controllers.yml
- Execution: Changes into the CartPole simulation directory and runs run_cartpole_gui.py

Driver/control.py: (Run time entry point)

- Configures the execution environment (module paths, working directory, TensorFlow CPU only + XLA)
- Instantiates the CartPole model and sets its controller timestep from global settings
- Creates and runs the PhysicalCartPoleDriver to control real hardware in real time
- Depends on: TensorFlow, globals.py, CartPoleSimulation, and DriverFunctions
- Acts as an entry point to physically control the cartpole

Simulation/GUI

Run_cartpole_gui.py:

- Starts the CartPole GUI from main folder

Running Simulation:

- Can edit a bunch of settings (initial pos, initial angle, latency), sim
- Can save recordings of experiments as CSV files

Hardware Conversion Overview

Run_vitis_cartpole.sh: This script is a launcher for Xilinx Vitis 2020.1 that runs the existing generate_vitis_project.tcl

- Can modify the file located at Driver/CartPoleSimulation/SI_Toolkit_ASF/config_hls.yml
- Can set factors (Precision, strategy, reuse)
- Required to set up PATH to Vitis

generate_vitis_project.tcl:

- Automates Vitis workspace setup to create hardware, BSP, firmware app, link source code, and build the CartPole firmware with correct math library settings
- Generates FSBL and BOOT.bin by combining the firmware ELF, FPGA bitstream, and Zynq bootloader for deployment

Conversion:

- Start by running Convert_with_hls4ml.py
 - This creates HLS4ML folder based on YAML configurations

Convert_with_hls4ml.py:

- Reads an HLS conversion configuration from SI_Toolkit_ASF/config_hls.yml
- Loads a trained network (Keras/TensorFlow model) using SI_Toolkit's get_net()
- Copies the original model directory to another location (so the HLS build has a local workspace)
- Converts the model to an hls4ml model, plots it
- Reads and summarizes Vivado reports from the output directory

globals.py: (NOTE: Has a lot of TODO's and FIXME's)

- Defines a lot (hardware target, controller, controller period, motor calibrations, unit conversion factors, runtime toggles, serial communication settings, few helped functions)
- Uses some parameters from cartpole.cartpole_parameters
- Used by DriverFunctions, CartPoleSimulation, Controllers, GUI, DataAnalysis

interface.py:

- Defines PC to firmware serial communication layer
- Saves experiment results to CSV via pandas

create_symlinks_cartpole.sh:

- Recursively copies source directory into IDE project folders (files only)
- Supports multiple firmware targets (STM, Zynq, NeuralImitator) by switching the source/destination directory mappings
- Depends on: bash, realpath, ln, and project source layout, used to sync firmware code into CubeIDE/Vitis projects without duplication

generate_bitstream.tcl: Builds the Zynq CartPole hardware design in Vivado, generates the FPGA bitstream, and exports the hardware platform (.xsa) for Vitis